Table of Contents

Introduction & Goal 2

Version 1 - Preliminary Experiment 4

Version 2 - Better State Implementation 5

Version 3 - SciPy Optimization 7

Version 4 - Evolutionary Optimization 9

Introduction & Goal

The project uses Reinforcement Learning to train a "bacterium" as the agent to find a food source with as few steps as possible in an environment with a concentration gradient. The bacterium cannot use its location coordinates, and has to rely on changes in its local concentration to find the food source. The goal is to investigate how changing different parameters including the learning rate, the discount factor, the exploration rate, and the reward function would affect the agent's success, and how to find optimize them efficiently using evolutionary algorithms.

Environment

- 2D grid [-4, 4] for each axis
- Food source located at (0, 0), and releasing a chemical attractant
- Fixed concentration: chemical concentration at each (x, y) which we call $c(x, y)$ is constant and is calculated as

$$c(x, y) = \frac{1}{1 + \sqrt{x^2 + y^2}}$$

Agent

- Find the food source with as few steps as possible
- Q-learning:

$$Q(s, a) := Q(s, a) + \alpha \left(R(s, a) + \gamma \max_{a'} Q(s', a') - Q(s, a) \right)$$

  - $a \in \{0,1,2,3\}$ (up, down, right, left) - agent's allowed actions
  - $s =$ agent's possible states
  - $R(s, a)$ = reward function

- γ = discount factor
- ϵ = exploration rate
- α = learning rate

Success of the Agent

Let the function $f(\theta)$ return the average number of excess steps (*agent's steps* $-\left(|x|+|y|\right)$) given the input $\theta = \left(\alpha, \gamma, \epsilon, T, t, c, d\right)$ where α, γ, ϵ are the Q-learning algorithm's hyperparameters and T, t, c, d are parameters for the reward function such that

$$R = \begin{cases} +T & \text{if found target} \\ +(t+c) & \text{if concentration increased} \\ +(t+d) & \text{if concentration decreased} \end{cases}$$

To calculate $f(\theta)$, we will create an agent with these parameters and train it with 100 episodes. Then, we will obtain a sample list in random order of all possible locations in the environment, repeating each location $N = 100$ times. We will have the agent start from each location and find the food source. The agent will be allowed to "learn"and revise its Q-table while navigating, but it will not be allowed to "explore" (i.e. choose a random action at times indicated by the exploration rate ϵ). Across all these trials, we will find the average number of excess steps the agent takes for a random starting location.

Version 1 - Preliminary Experiment

Description

The agent only knows whether its concentration increases or decreases after each action. Therefore, the state at time t can be stored as $s_t = (\Delta c, a_{t-1})$, where $\Delta c \in \{0,1\}$, denoting a decrease and an increase in concentration, respectively. Hence, the agent has 9 states (with 1 being its initial state where a_{t-1} is undefined).

Results

- After some trial and error, the following set of parameters were found to be fairly optimal

$$\theta = (\alpha, \gamma, \epsilon, T, t, c, d) = (1.0,\ 0.2,\ 0.15,\ 1000, -1,\ 0, -9)$$

- Overall, however, it does not work very well, because it cannot uniquely identify where it is in the environment solely based on its state.
- According to figure 1, this version fails to reach the target in at most 100 steps when calculating $f(\theta)$ ~86.7% of the time:

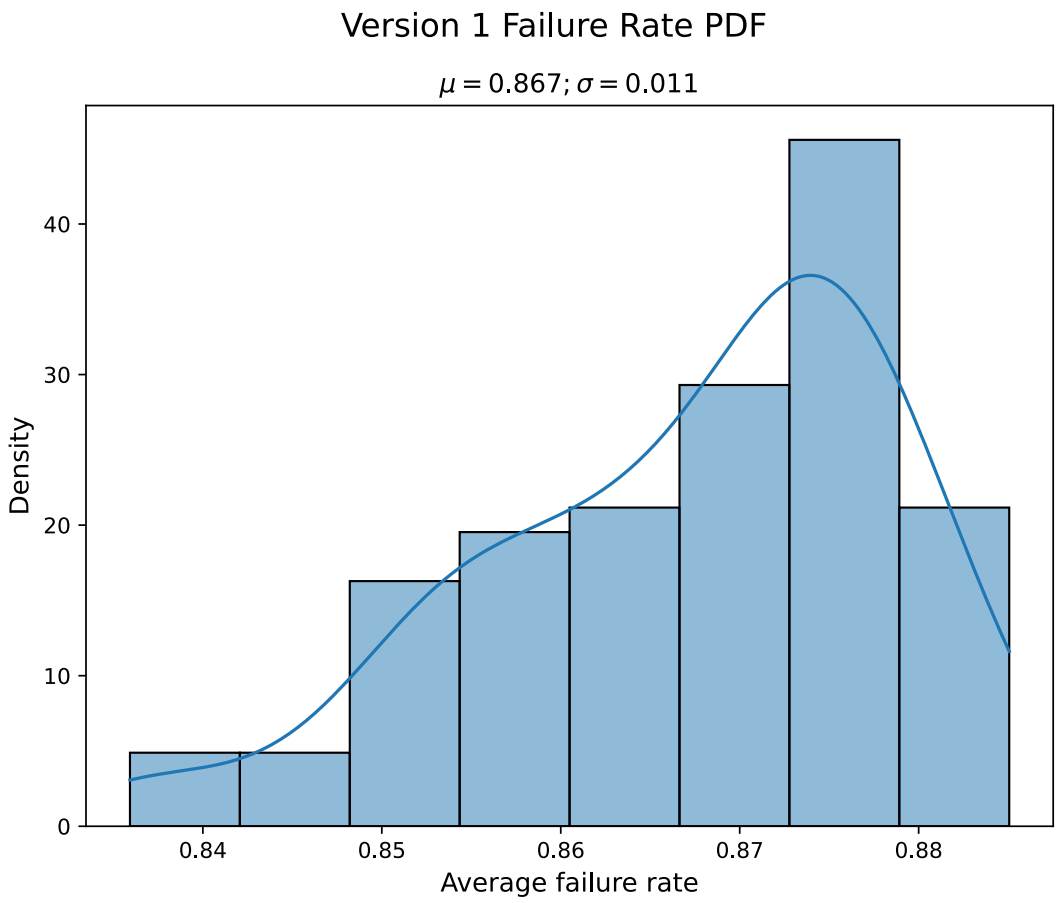


Figure 1: PDF of Version 1's Average Failure Rate

Next steps

- Make the state dependent on $(c_t,\ c_{t-1},\ a_{t-1})$

Version 2 - Better State Implementation

Description

Now the agent knows its current concentration, its previous concentration, and its previous action. This would allow it to better identify where it is relative to the food source. The same $\theta_2 = \theta$ was used for this version.

Results

This version had a significant improvement compared to the previous one. The failure rate went to 0.0 and, as seen in figure 2, the mean of average excess steps centered around $f\left(\theta_2\right) = 2.428$.

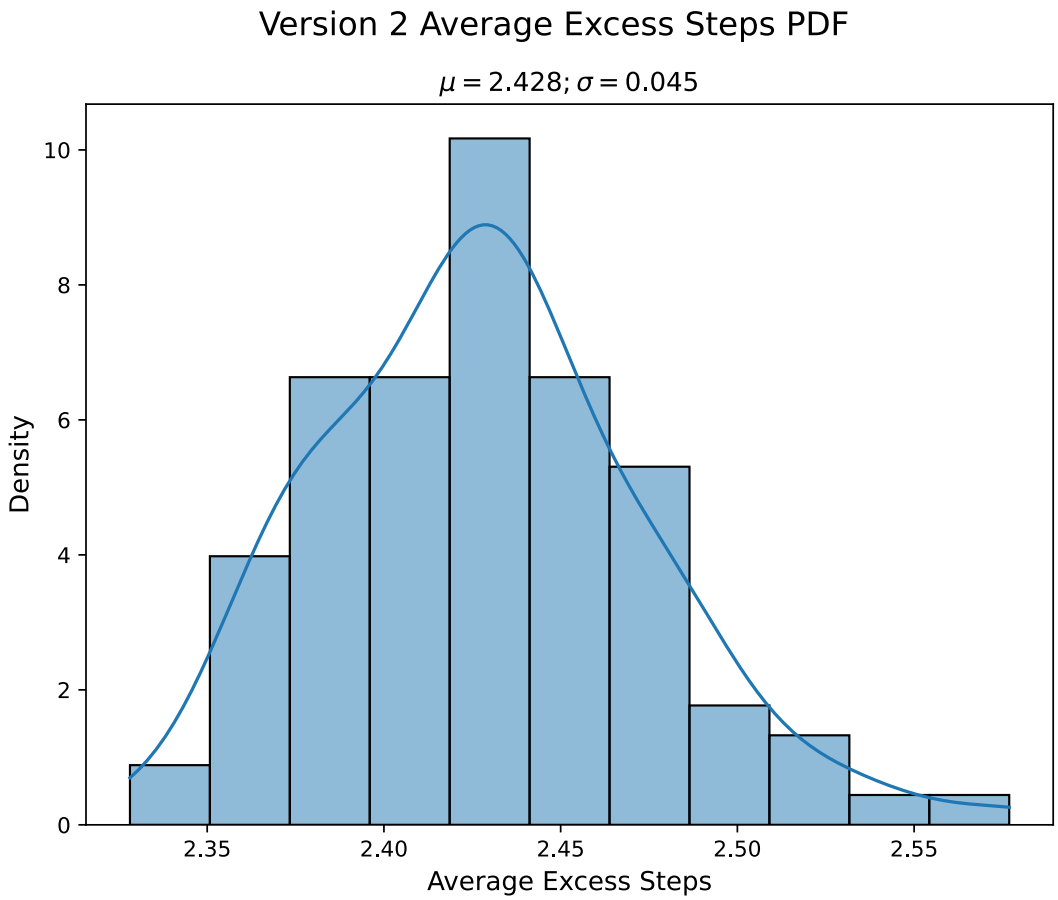


Figure 2: PDF of Version 2's Average Excess Steps $P\left(f\left(\theta_2\right)\right)$

Next steps

Next, we will proceed to optimize the values of $\alpha, \gamma, \epsilon, T, t, c, d$ computationally and investigate how $P\left(f\left(\theta_3\right)\right)$ compares with $P\left(f\left(\theta_2\right)\right)$.

Version 3 - SciPy Optimization

Description

We will utilize SciPy Optimize to find a value for θ_3 that would minimize f. We will use θ_2 as our initial guess and use the Nelder-Mead method for finding the minimum, as f is a 7-dimensional, highly stochastic function. Everything else regarding the agent's states and actions remains the same.

Results

After the first tuning, the following θ was returned as optimal:

$$(\alpha, \gamma, \epsilon, T, t, c, d) = (0.958,\ 0.204,\ 0.148,\ 1026.359, -1.016,\ 1.842, -8.772)$$

However, it only offered a slight improvement over version 2. Therefore, that set of parameters was reused as an initial guess and re-optimized to get

$$\theta_3 = (0.962,\ 0.205,\ 0.149,\ 1044.287, -0.990,\ 1.855 \times 10^{-4}, -8.830)$$

As seen in figure 3, θ_3 clearly shifts $P(f_3)$ to the left compared to $P(f_2)$, with a smaller mean (2.295 vs 2.428) and standard deviation (0.0043 vs 0.0045).

At the same time, it is worth noting that this process was rather computationally heavy, with each single optimization taking around 4 minutes to complete.

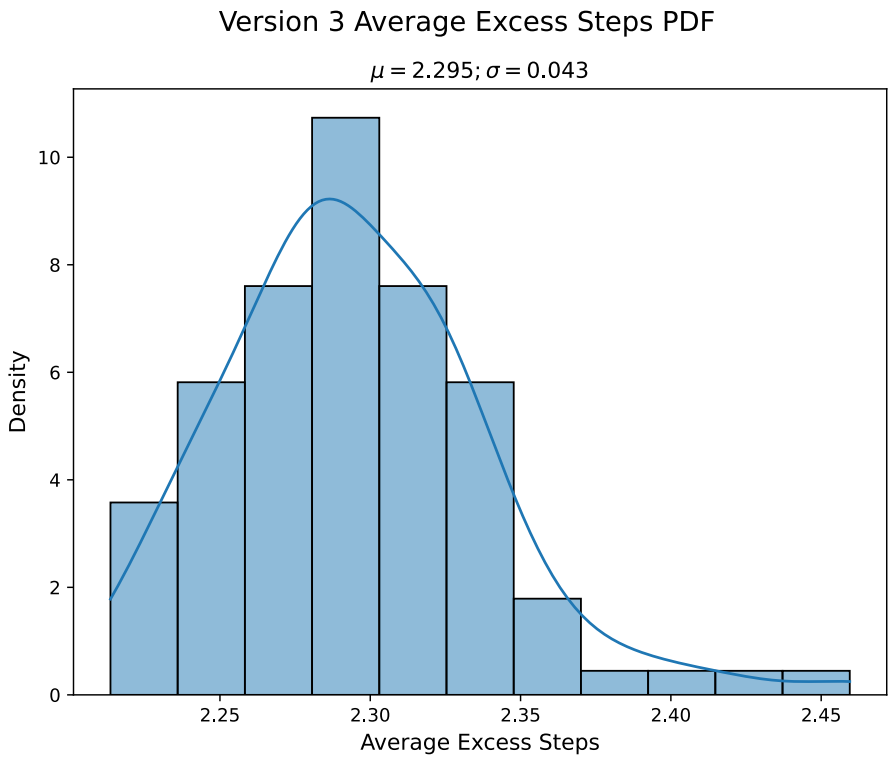


Figure 3a: PDF of Version 3's Average Excess Steps $P\left(f\left(\theta_3\right)\right)$

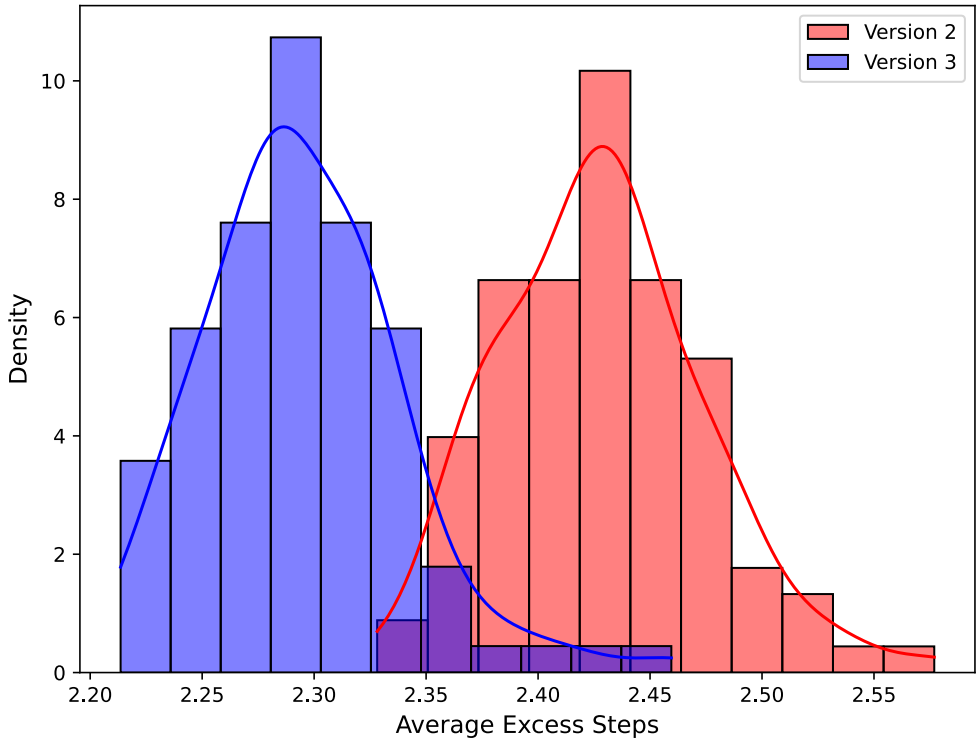


Figure 3b: A comparison of version 2 and 3's PDFs $P\left(f_2\right)$ vs $P\left(f_3\right)$

Version 4 - Evolutionary Optimization

Description

We will consider this problem from an evolutionary standpoint. Suppose that each agent with a particular set of hyperparameters is a "species," and that each hyperparameter is a "gene" in the organism. When the surviving species reproduce, each of these genes can undergo mutation with a certain rate. When a gene mutates, its value is selected at random from the following domains:

$$\alpha \in [0.0,\ 1.0]$$

$$\gamma, \epsilon \in [0.0,\ 1.0)$$

$$T, t, c, d \in [-1000,\ 1000]$$

As a starting point, assume that each episode, we start with 20 agents: 10 agents with $\theta = \theta_2$ and 10 with $\theta = (0.5,\ 0.5,\ 0.5,\ 10, -1,\ 0,\ 0)$. Let's give each hyperparameter a mutation rate of 0.25. Each agent will undergo 100 training episodes as before, and then, start from a random location in the environment and try to reach the food source (with learning and without exploration). We only keep the 5 best organisms (the ones with the smallest number of excess steps), and we assume that the rest were outcompeted in competition and eliminated. We assume the agents that fail to find the food source within 100 steps have an infinite excess steps number (since we are certain that the 10 agents with θ_2 will not fail, we can simply let the unfit agents with infinite excess steps get eliminated). Then, each of the 5 remaining agents can produce 3 offsprings (that may have a different set of genes due to mutation) to get a new set of 20 agents. We can repeat this

evolution cycle 100 times and output the 5 best θ_E that remain. We will then compare $P\left(f_E\right)$ with $P\left(f_2\right)$ and $P\left(f_3\right)$.

Results

Table 1 shows the 5 remaining θ at the end of the 100 evolution/competition cycles.

	α	γ	ϵ	T	t	c	d
θ_{E_1}	0.014	0.293	0.005	571.76	314.87	447.04	-617.08
θ_{E_2}	0.858	0.293	0.415	571.76	314.87	-326.51	-869.57
θ_{E_3}	0.014	0.293	0.415	571.76	314.87	-326.51	-617.08
θ_{E_4}	0.14	0.293	0.415	571.76	314.87	-326.51	-617.08
θ_{E_5}	0.14	0.418	0.415	571.76	314.87	235.72	-617.08

Table 1: θ_{E_1}- θ_{E_5} remaining at the end of 100 evolutionary cycles

Figure 4 shows $P\left(f\left(\theta_{E_1}\right)\right)$ (4a) and how it compares with $P\left(f_2\right)$ and $P\left(f_3\right)$ (4b).

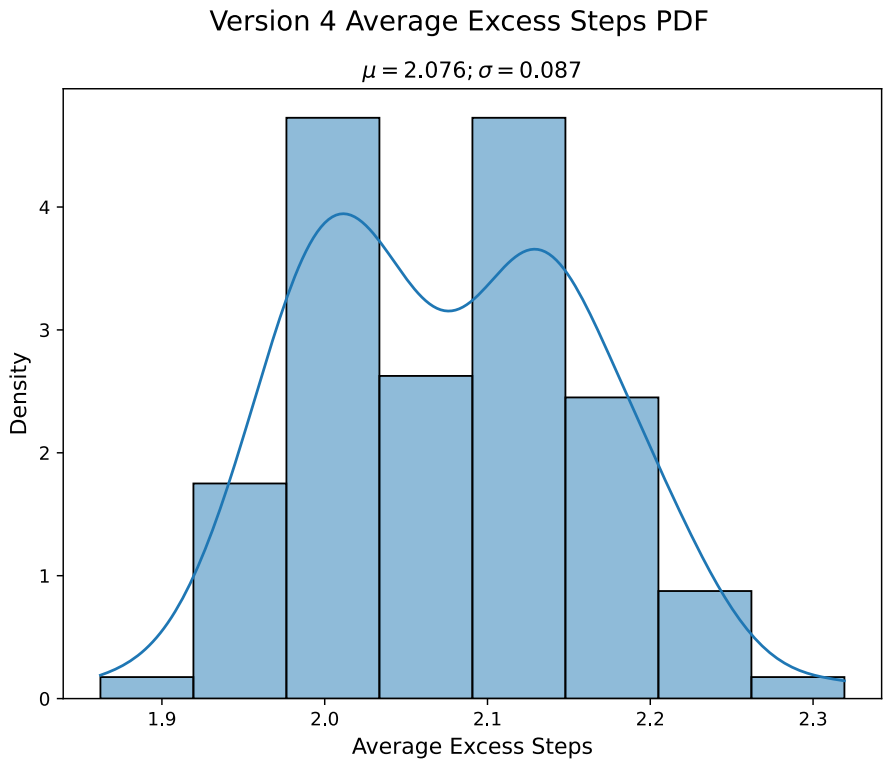


Figure 4a: PDF of $f\left(\theta_{E_1}\right)$

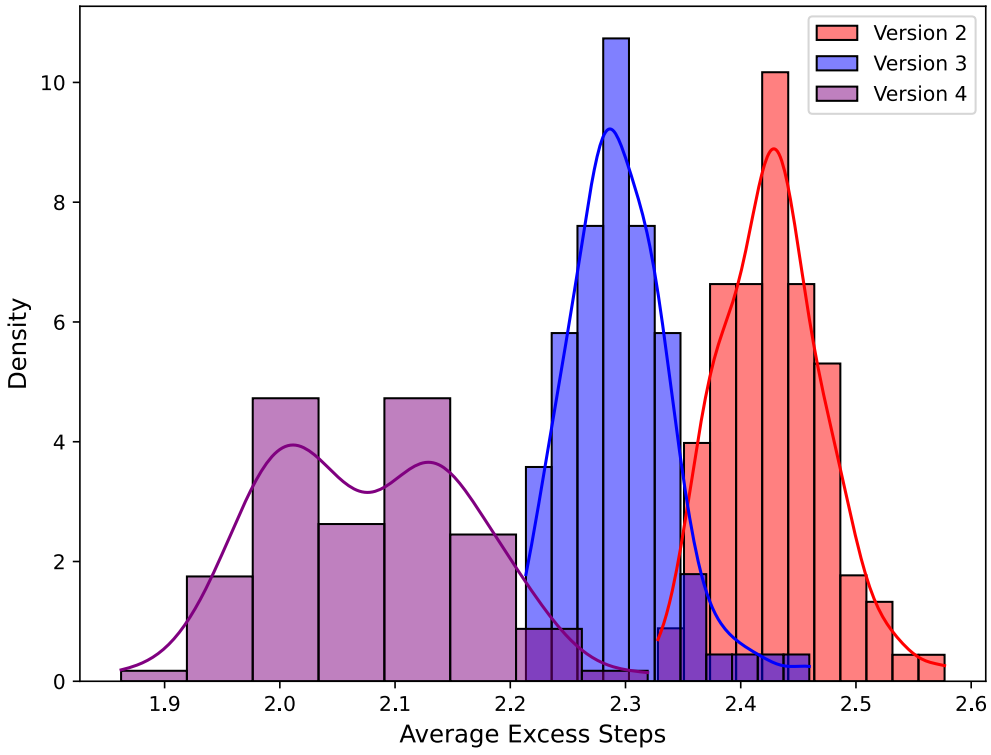


Figure 4b: Comparison of the 4 PDFs

As seen, the evolutionary optimized hyperparameters are clearly performing better than the previous two. The mean of $f\left(\theta_{E_1}\right)$, μ_{E_1} is below both μ_2 and μ_3 by over 0.2, which is about 10%, and $P\left(f_{E_1}\right)$ is clearly shifted to the left compared to $P\left(f_2\right)$ and $P\left(f_3\right)$. It is also worth noting that the hyperparameters' values are quite different from what was initially assumed. Another interesting observation is that $P\left(f_2\right)$ and $P\left(f_3\right)$ are both unimodal and skewed right, while $P\left(f_{E_1}\right)$ is bimodal and less skewed.

We can see that evolutionary optimization has advantages over more traditional optimizations like SciPy's Nelder-Mead, including:

1. **Less computational weight & faster runtime**: while each SciPy optimization took around 4 minutes (with a total of 8 minutes spent for undergoing two episodes to find parameters that resulted in a meaningful improvement), the entire evolutionary optimization process over all cycles took less than 2 minutes to complete.
2. **Broader range of exploration**: while the Nelder-Mead method's calculated values stayed very close to the original θ_2, the evolutionary optimization explored a wider variety of θ and found a set completely different from what was originally supposed which proved to be more beneficial.

Overall, it is evident that the evolutionary approach was able to discover better hyperparameters in a significantly shorter runtime and with a lot less computational power than SciPy. This property is especially important for

settings that are highly stochastic and involve a high number of dimensions, which was the case for our system.